

Technology Stack

Date	29 june 2026
Team ID	XXXXXX
Project Name	CreditCard Approval Prediction
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements. Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (e.g., React, Angular, HTML/CSS)	HTML5, CSS3	<i>Chosen to rapidly construct a clean, modern, and highly responsive user evaluation form framework. Bootstrap 5 eliminates the need for complex, heavy custom styling while ensuring grid input elements adapt properly across multiple desktop monitor aspect ratios.</i>

2	Backend / Server-Side <i>(e.g., Node.js, Python/Django, Java)</i>	<i>Python 3, Flask Web Framework</i>	<i>Python provides a native ecosystem for executing complex machine learning pipelines. Flask was chosen because it is an exceptionally lightweight, unopinionated WSGI server framework. It allows us to set up dynamic request POST hooks and run synchronous model calculations efficiently without massive boilerplate code.</i>
3	Database / Data Storage <i>(e.g., MySQL, MongoDB, PostgreSQL)</i>	<i>Pandas DataFrame, Binary File System (.pkl)</i>	<i>For the core historical analytical dataset, structured local CSV sheets (application_record.csv and credit_record.csv) are ingested dynamically into memory-efficient Pandas DataFrames. The final state of our trained decision boundary parameters is serialized as a lightweight binary file (model.pkl) using Python's built-in pickle library, bypassing the configuration overhead of external database servers.</i>

4	Cloud / Hosting / Deployment <i>(e.g., AWS, Azure, Heroku, Docker)</i>	<i>Localhost Environment (Development Stack)</i>	Deployed on a localized WSGI development sandbox environment (http://127.0.0.1:5000/) for immediate internal system testing, code debugging, and verification loops. This enables fast, iterative updates to the codebase without introducing cloud runtime costs or latency variations.
5	Version Control & CI/CD <i>(e.g., GitHub, GitLab, Jenkins)</i>	<i>[Specify technology]</i>	Chosen to establish stable codebase version control, track development updates across step transitions, safeguard source code assets, and maintain a historical repository trail of code edits across our data processing and model build steps.
6	Third-Party APIs / Other Tools <i>(e.g., Stripe, Twilio, Google Maps)</i>	<i>Scikit-Learn, NumPy, Seaborn / Matplotlib</i>	Scikit-Learn supplies production-grade implementations of predictive estimators (RandomForestClassifier, DecisionTreeClassifier, LogisticRegression) along with essential transformation utilities (LabelEncoder). NumPy optimizes matrix vector transformations from input arrays, and Seaborn powers essential exploratory diagnostic plots (like target count charts).

